EASWARI ENGINEERING COLLEGE
RAMAPURAM, CHENNAI
AUTONOMOUS

SMART REPLY

A PROJECT REPORT

Submitted by

RAM KUMAR (319625205224)

SAM ROHITH (31062520541)

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

EASWARI ENGINEERING COLLEGE, RAMAPURAM
(AUTONOMOUS)

ANNA UNIVERSITY: CHENNAI 600 025

NOV - DEC 2026

BONAFIDE CERTIFICATE

Certified that this project work under the course 2321CSC304R **OBJECT ORIENTED PROGRAMMING USING JAVA titled "SMART REPLY" is the Bonafide work of** "RAM KUMAR (319625205224), SAM ROHITH (31062520541)" who carried out the project work under my supervision.

Mrs.S.Saranya M.E.,(Ph.D.,)

SUPERVISOR

Assistant Professor

Department of Information Technology

Easwari Engineering College

Ramapuram, Chennai-600 089

Dr Ananthi N, M.E., Ph.D.,

HEAD OF THE DEPARTMENT

Professor

Department of Information Technology

Easwari Engineering College

Ramapuram, Chennai-600 089

Submitted for the project work examination held on …… .

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ABSTRACT

Email remains the primary channel of professional, academic and business communication, and the number of messages that a person receives every day continues to grow. Reading each message, deciding what to say and typing a well-structured response consumes a considerable share of working time, delays important replies and lowers productivity. Existing email clients offer only short, fixed quick-reply phrases or rule-based auto-responders such as out-of-office messages, which cannot understand the context of a message or adapt to the tone and intent of the sender. To address these limitations, this project proposes SMART REPLY, an AI-powered automatic email reply website that reads an incoming mail, understands its intent and generates a complete, context-aware reply using a Large Language Model (LLM). The website provides a home page, secure registration and login pages, a dashboard showing the connected mailbox, an AI reply composer with selectable tone (formal, friendly, concise or detailed) and a reply history module. The mail integration module retrieves messages through secure OAuth 2.0 based access, the AI module builds a structured prompt from the email thread and the user's preferences, and the review module allows the user to edit, approve and send the generated reply, so that a human always remains in control. Security is ensured through password hashing, JSON Web Token (JWT) based sessions and encrypted storage of mailbox tokens. The proposed framework reduces the time spent on routine correspondence, improves the consistency and professionalism of replies, and converts email handling from a manual chore into an assisted, efficient workflow.

ACKNOWLEDGEMENT

We thank the Almighty GOD for the abundant blessings showered on us throughout the course of this project. We extend our deepest love and gratitude to our dear parents, who built up our career and supported us at every stage of our life.

We thank our management and our Principal Dr. P. Deiva Sundari for providing the opportunities and facilities that helped us in our career development and in the successful completion of this project.

We feel indebted to the Head of the Department Dr. N. Ananthi, Professor, Department of Information Technology, Easwari Engineering College, for her constant encouragement, which has sustained our labour and efforts in developing the SMART REPLY website.

We express our deepest gratitude to our internal guide Mrs. S. Saranya, Assistant Professor, Department of Information Technology, Easwari Engineering College, for her valuable guidance, ideas and support during the design and implementation of the project.

We would like to thank all other faculty members of the Department of Information Technology for their help and advice throughout this project.

Finally, we are thankful to all our friends and all others who encouraged us, tested our work and helped us in completing this project.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	3
	ACKNOWLEDGEMENT	4
	LIST OF TABLES	6
	LIST OF FIGURES	7
	LIST OF ABBREVIATIONS	8
1	**INTRODUCTION**	**1**
	1.1 OVERVIEW	1
	1.1.1 CAUSES OF EMAIL OVERLOAD	2
	1.1.2 EFFECTS OF DELAYED EMAIL RESPONSES	2
	1.2 OBJECTIVE	3
	1.3 CHALLENGES IN THE DOMAIN	4
2	**RELATED WORK**	**6**
	2.1 LITERATURE REVIEW	6
	2.2 EXISTING TECHNOLOGY	7
	2.3 INFERENCE OF LITERATURE REVIEW	8
	2.4 EXTRACTION FROM LITERATURE REVIEW	10
3	**SYSTEM ANALYSIS**	**11**
	3.1 PROBLEM DEFINITION	11
	3.2 PROPOSED SOLUTION	11
	3.3 SOFTWARE COMPONENTS	13
	3.4 HARDWARE COMPONENTS	15
	3.5 USE CASES	15
4	**IMPLEMENTATION AND OUTPUT SCREENSHOTS**	**18**
	4.1 EXPERIMENTAL SETUP	18
	4.2 SYSTEM ARCHITECTURE	19
	4.3 MODULE IMPLEMENTATION DETAILS	22
	4.3.1 USER AUTHENTICATION MODULE	22
	4.3.2 MAIL INTEGRATION MODULE	24
	4.3.3 AI REPLY GENERATION MODULE	25
	4.3.4 REPLY REVIEW AND SEND MODULE	27
	4.3.5 WEB INTERFACE AND DASHBOARD MODULE	29
	4.4 OUTPUT AND SCREENSHOTS	30
5	**CONCLUSION AND FUTURE ENHANCEMENTS**	**35**
	5.1 CONCLUSION	35
	5.2 FUTURE ENHANCEMENTS	36
	APPENDIX 1: SAMPLE CODE	**38**
	REFERENCES	**45**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Inference of Literature Review	9
3.1	Summary of Software Components	15
3.2	Hardware Requirements	15
4.1	Database Tables	21
4.2	Main REST API Endpoints	21
4.3	Tone Options and Prompt Instructions	27
4.4	Test Cases and Expected Behaviour	34

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	System Architecture of SMART REPLY	19
4.2	Reply Generation Workflow	20
4.3	Home Page	30
4.4	Login Page	31
4.5	Dashboard and Inbox View	31
4.6	AI Reply Composer with Generated Draft	32
4.7	Reply History Page	33
4.8	Settings Page	33

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
HTTP(S)	HyperText Transfer Protocol (Secure)
IMAP	Internet Message Access Protocol
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
NLP	Natural Language Processing
OAuth	Open Authorization
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
UI	User Interface

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Electronic mail remains the most widely used channel for professional, academic and business communication. Employees, students, freelancers and customer support teams exchange hundreds of messages every week, and a large portion of these messages need a reply that is accurate, polite and timely. Reading every message, understanding what the sender wants and typing a well-structured response is a repetitive activity that consumes a considerable share of the working day. When the volume of mail increases, replies are delayed, important messages are overlooked and the quality of writing suffers.

An automatic reply system attempts to reduce this burden. Traditional auto-responders, such as out-of-office messages, send the same fixed text to every sender and make no attempt to understand the content of the incoming mail. Quick-reply buttons offered by some email clients provide only a few short phrases such as “Thank you” or “Sounds good”. Neither approach can answer a specific question, confirm a meeting time or politely decline a request. Recent advances in Natural Language Processing (NLP) and Large Language Models (LLMs) now make it possible to read an email in context and write a complete, meaningful response in a few seconds.

SMART REPLY is an AI-powered automatic email reply website developed to address this need. The website allows a user to register, log in securely, connect a mailbox and view incoming messages on a dashboard. For every selected message, the system analyses the subject, body and previous conversation, and generates a ready-to-send reply using an AI model. The user can choose the tone of the reply, such as formal, friendly or concise, edit the draft if required and send it directly from the website. The project therefore combines a user-friendly web interface, a secure backend, mail integration and an AI reply engine into a single, practical application.

The design of SMART REPLY follows a human-in-the-loop principle. The AI prepares the draft, but the user reviews and approves it before it is sent. This keeps the user in control of what is communicated in his or her name, reduces the risk of incorrect replies and builds trust in the system. By focusing on speed, personalization and safety, SMART REPLY

aims to turn email handling from a time-consuming chore into an assisted and efficient workflow.

1.1.1 CAUSES OF EMAIL OVERLOAD

a) High Volume of Incoming Mail: Newsletters, notifications, internal circulars, customer queries and project updates arrive continuously. Even a careful user finds it difficult to read and answer every message on time, and the inbox gradually becomes a backlog of pending replies.

b) Repetitive and Routine Queries: A large share of business email consists of questions that have been answered many times before, such as requests for meeting timings, document status, pricing details or confirmation of receipt. Writing these answers again and again wastes effort that could be spent on creative or analytical work.

c) Time Pressure and Context Switching: Users receive mail while they are working on other tasks. Switching between work and email breaks concentration, and rushed replies are more likely to contain mistakes, missing details or an inappropriate tone.

d) Language and Tone Difficulties: Many users communicate in a language that is not their first language, or they are unsure how formal a message should be. Composing a polite and professional reply for a senior manager, a client or a faculty member takes extra time and thought.

e) Lack of Intelligent Tools: Basic filters, folders and templates only organise mail; they do not help in writing the response. Without an intelligent assistant that understands the message, the user must start every reply from a blank screen.

1.1.2 EFFECTS OF DELAYED EMAIL RESPONSES

a) Loss of Productive Time: Studies of knowledge work consistently show that a significant part of the working week is spent reading and writing email. Time spent on routine replies is time taken away from planning, development, teaching or decision making.

b) Delayed Responses and Missed Opportunities: A late reply to a customer enquiry, a job offer, an interview invitation or a client request can lead to lost business, lost opportunities and a negative first impression.

c) Inconsistent Quality of Communication: When different people answer similar questions in different words, customers receive inconsistent information. Spelling errors, unclear sentences and an unsuitable tone also reduce the credibility of the sender.

d) Stress and Reduced Focus: A constantly growing list of unanswered messages creates mental pressure. Users feel obliged to check their inbox repeatedly, which further interrupts their primary work and affects well-being.

e) Reduced Customer and Stakeholder Satisfaction: In service-oriented organisations, quick and helpful replies are a major part of the customer experience. Slow or generic responses damage the reputation of the organisation and make it harder to retain customers.

1.2 OBJECTIVE

The primary objective of this project is to develop SMART REPLY, an AI-powered automatic email reply website that helps users respond to incoming mail quickly, accurately and in an appropriate tone. To achieve this, the project focuses on building a complete web application that integrates user management, secure mailbox access, natural language understanding and automated reply generation.

The project aims to design a responsive website consisting of a home page that explains the service, registration and login pages for secure access, a dashboard that lists incoming mail, and a reply composer that displays the AI-generated draft. A secure backend is developed to authenticate users, to retrieve mail from the connected account through authorised access, and to store user preferences and reply history. An AI reply engine is then integrated, in which the email content and conversation history are converted into a structured prompt and sent to a Large Language Model, which returns a context-aware reply in the tone selected by the user.

Another key objective is to keep the user in control. The system provides a review and edit step so that every reply can be corrected before it is sent, and it keeps a history of generated and sent replies for later reference. The project also aims to protect user data by using password hashing, token-based sessions and encrypted storage of mailbox credentials. Finally, the project seeks to provide a simple dashboard that summarises how many replies were generated, accepted and sent, so that users can see the time saved by the system and gain confidence in its use.

1.3 CHALLENGES IN THE DOMAIN

a) Understanding Context and Intent

An email is rarely a single isolated sentence. It may be part of a long thread, refer to earlier decisions, contain several questions or hide the real request behind polite language. The system must correctly identify what the sender is asking and must not confuse quoted older messages with the latest request. Wrong interpretation leads to an irrelevant or misleading reply.

b) Accuracy, Hallucination and Trust

Language models generate fluent text, but fluency is not the same as correctness. A model may invent a date, a price or a commitment that the user never made. Since a reply is sent in the name of a real person, such errors can have serious consequences. The system therefore needs careful prompt design, clear instructions not to invent facts and a mandatory human review step.

c) Tone and Personalization

The same message may need a formal answer when it comes from a client and a friendly answer when it comes from a colleague. Replies should also reflect the writing style, signature and preferences of the user. Achieving this level of personalization without asking the user to fill long forms is difficult.

d) Privacy and Security of Mail Data

Emails contain personal, financial and confidential information. Accessing a mailbox, storing tokens and sending content to an external AI service require strict security practices, user consent and minimal data retention. Failure to protect this data would destroy user trust and may violate data protection rules.

e) Latency, Cost and Reliability

Calls to an AI service take time and may fail because of network problems or rate limits. A good user experience requires the draft to appear within a few seconds, and the system must handle timeouts, retries and temporary outages gracefully while keeping the cost of each request under control.

f) Usability and Integration

The interface must be simple enough for non-technical users and must work smoothly on different screen sizes. Integrating with real mail providers, which have their own authentication flows, message formats and usage limits, adds further complexity to the design.

CHAPTER 2

RELATED WORK

2.1 LITERATURE REVIEW

Automatic response generation for email has been studied for more than a decade, with researchers focusing on both the quality of suggestions and the practicality of deploying them at scale. K. Kannan et al. [1] described the original Smart Reply system used in a widely deployed email application, in which an encoder-decoder neural network read an incoming message and scored a fixed set of short candidate responses. Their work introduced important ideas such as a triggering model that decides whether suggestions should be shown at all, and clustering of candidate responses to keep the suggestions diverse. M. Henderson et al. [2] later proposed a more efficient response suggestion approach that ranks candidate replies using a feed-forward network and a dual-encoder structure, showing that fast retrieval-style ranking can achieve good quality with lower computational cost. M. X. Chen et al. [3] extended the idea from replying to writing assistance in their work on real-time sentence completion, highlighting the strict latency requirements and the importance of using the subject and earlier messages as context.

The quality of automatic text generation depends strongly on the underlying sequence models. I. Sutskever et al. [4] demonstrated that recurrent networks can map one sequence to another, which became the foundation of early reply generation systems. S. Hochreiter and J. Schmidhuber [5] introduced Long Short-Term Memory networks to overcome the difficulty of learning long-range dependencies. A. Vaswani et al. [6] then proposed the Transformer architecture based on self-attention, which allowed much faster training and better handling of long text. J. Devlin et al. [7] showed that bidirectional pre-training on large text corpora produces models that understand sentence meaning very well, which is useful for tasks such as intent detection and classification of incoming mail.

Large generative language models have changed the way reply systems are built. T. B. Brown et al. [8] showed that a sufficiently large language model can perform new tasks from only a few examples placed in the prompt, without any additional training. This property makes it possible to build a reply assistant by simply describing the task, the tone and a few sample replies. L. Ouyang et al. [9] explained how aligning language models with human

feedback makes them follow instructions more reliably and reduces unwanted outputs, which is essential for professional communication. P. Lewis et al. [10] introduced retrieval-augmented generation, in which the model is supplied with relevant documents at generation time to reduce invented facts. T. Wolf et al. [11] contributed an open-source library that made pre-trained Transformer models easy to use in practical applications.

A web-based reply system also depends on secure and standard web technologies. D. Hardt [12] specified the OAuth 2.0 authorisation framework, which allows an application to access a user's mailbox without ever receiving the user's password. M. Jones et al. [13] defined the JSON Web Token format used to carry authenticated session information between the browser and the server. N. Provos and D. Mazières [14] proposed an adaptive password hashing scheme, now widely known as bcrypt, that remains resistant to brute-force attacks as hardware becomes faster. R. T. Fielding [15] described the REST architectural style, which guides the design of the HTTP-based interface between the front end and the back end of modern websites.

Overall, the literature shows that early systems offered only short canned suggestions, whereas current language models can write complete and context-aware replies. However, most published work concentrates on algorithms inside large commercial products. Little attention has been given to a simple, open and user-controlled website that combines secure mailbox access, tone selection, human review and reply history. These gaps form the basis of the SMART REPLY project.

2.2 EXISTING TECHNOLOGY

Handling incoming email has traditionally been a manual activity supported by simple tools. Rules and filters move messages into folders, mark them as important or delete them, but they do not help in writing the answer. Out-of-office auto-responders send a single pre-written message to every sender during a defined period. Canned responses or templates allow a user to insert saved paragraphs, yet the user must still choose the right template and adjust the details by hand.

Commercial email platforms introduced the first intelligent assistance. Suggested replies offer three short responses at the bottom of a message, and predictive text completes sentences while the user types [1][3]. These features are fast and convenient, but they are limited to short phrases, offer very little control over tone, and are available only inside the

provider’s own application. A user who needs a detailed answer to a specific question still has to write most of the reply personally.

With the arrival of generative language models [8][9], a new class of tools appeared that can draft complete emails from a short instruction. General-purpose chat assistants can write replies when the user pastes the message and describes what is needed, but this involves copying text between applications, repeating instructions every time and exposing the whole conversation manually. Dedicated email assistants and browser extensions simplify this process, yet many of them are closed products with fixed subscriptions, limited customisation and little transparency about how mail data is handled.

The technology required to build a better solution is now widely available. Mail can be accessed securely through provider interfaces such as the Gmail API or the IMAP and SMTP protocols, with authorisation handled by OAuth 2.0 [12]. Back-end frameworks written in Java, such as Spring Boot, provide a robust platform for REST services [15], authentication with signed tokens [13] and database access through the Java Persistence API. Passwords can be stored safely using adaptive hashing [14]. Language models can be consumed as a service through HTTP interfaces, and libraries such as the Transformers library [11] allow experimentation with open models. Techniques such as retrieval-augmented generation [10] can further ground the replies in information supplied by the user.

In summary, the evolution of email technology has moved from manual handling and fixed templates to short machine-suggested phrases and finally to fully generated replies. SMART REPLY builds on these advances by packaging an LLM-based reply engine, secure mail integration and a human review workflow into an accessible website.

2.3 INFERENCE OF LITERATURE REVIEW

Title	**Contribution**
Smart Reply: Automated Response Suggestion for Email [1]	Presented a deployed system that suggests short replies using an encoder-decoder network, with a triggering model and diverse candidate selection.
Efficient Natural Language Response Suggestion for Smart Reply [2]	Proposed a fast feed-forward and dual-encoder ranking method that reduces the computation needed for response suggestion.

Title	Contribution
Gmail Smart Compose: Real-Time Assisted Writing [3]	Built a real-time writing assistant that completes sentences while typing, using the subject and previous messages as context under strict latency limits.
Sequence to Sequence Learning with Neural Networks [4]	Showed that recurrent networks can map an input sequence to an output sequence, forming the basis of early reply generation.
Long Short-Term Memory [5]	Introduced gated recurrent memory cells that learn long-range dependencies in text.
Attention Is All You Need [6]	Proposed the Transformer architecture based on self-attention, enabling faster training and better handling of long text.
BERT: Pre-training of Deep Bidirectional Transformers [7]	Demonstrated bidirectional pre-training that yields strong language understanding for classification and intent detection.
Language Models are Few-Shot Learners [8]	Showed that large language models can perform new tasks from instructions and a few examples placed in the prompt.
Training Language Models to Follow Instructions with Human Feedback [9]	Aligned language models with human feedback so that they follow instructions more reliably and produce fewer unwanted outputs.
Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [10]	Combined document retrieval with text generation to ground the output in supplied information and reduce invented facts.
Transformers: State-of-the-Art Natural Language Processing [11]	Released an open-source library that makes pre-trained Transformer models easy to use in applications.
The OAuth 2.0 Authorization Framework [12]	Defined a standard for delegated access to user resources, such as a mailbox, without sharing the password.
JSON Web Token (JWT) [13]	Defined a compact, signed token format for carrying authenticated session claims between client and server.
A Future-Adaptable Password Scheme [14]	Proposed an adaptive, salted password hashing scheme that stays resistant to brute-force attacks as hardware improves.
Architectural Styles and the Design of Network-based Software Architectures [15]	Described the REST architectural style that guides the design of HTTP-based web service interfaces.

Table 2.1 Inference of Literature Review

2.4 EXTRACTION FROM LITERATURE REVIEW

A review of the existing literature on automated email response reveals a steady progression from retrieval of short canned replies to the generation of complete, context-aware messages. K. Kannan et al. [1] and M. Henderson et al. [2] demonstrated that a reply assistant can be deployed at very large scale if the system is fast, selective about when it offers a suggestion and careful to present diverse options. M. X. Chen et al. [3] showed that the subject line and the earlier messages in a thread are valuable context, a lesson that SMART REPLY applies by including the conversation history in every prompt.

Sequence models [4][5] and Transformer-based models [6][7] provide the technical foundation for understanding and generating text. Instruction-following language models [8][9] remove the need to collect a large training corpus for each new task, because the behaviour of the system can be controlled through the prompt. This is the approach adopted in this project, where the tone, the length and the writing rules are described in a structured instruction that is sent along with the email. Retrieval-augmented generation [10] suggests that supplying relevant facts, such as the user's signature, working hours or frequently used answers, reduces the chance of invented content.

Several studies and standards also emphasise safe deployment. OAuth 2.0 [12] enables mailbox access without storing the user's password, signed tokens [13] protect user sessions, and adaptive hashing [14] protects stored credentials. The REST style [15] provides a clean separation between the website and the server so that each can evolve independently.

Across the literature, three requirements appear consistently: replies must be relevant to the full context of the message, the generation process must be fast enough for interactive use, and the user must remain able to check and control the result. Existing work addresses these requirements mostly inside proprietary products. SMART REPLY brings them together in a single website that is simple to use, configurable in tone, and designed with security and human oversight as core principles.

CHAPTER 3

SYSTEM ANALYSIS

3.1 PROBLEM DEFINITION

Although email clients now offer suggested replies and predictive typing, existing solutions still leave important problems unsolved. Rule-based auto-responders send identical text to every sender and cannot understand the message. Quick-reply suggestions are limited to a few short phrases and cannot answer a detailed question. General-purpose AI chat tools can write good replies, but they require the user to copy the email into another application, explain the task every time and copy the answer back, which breaks the workflow and wastes the time that automation is supposed to save.

Many tools also provide little control over the tone, the length and the signature of the reply, so that the generated text sounds impersonal. Some tools send replies automatically without giving the user a chance to review them, which creates the risk of incorrect or inappropriate messages. In addition, users are often unsure how their mailbox data is stored and shared, and few systems provide a clear history of what was generated and sent. Finally, there is a lack of simple, secure and user-friendly websites that bring mailbox access, AI generation, human review and reply tracking together in one place. Addressing these gaps is essential for building a practical, trustworthy and efficient automatic email reply system.

3.2 PROPOSED SOLUTION

To address the limitations of the existing approaches, this project proposes SMART REPLY, an AI-based automatic email reply website that provides accurate, personalised and secure reply drafts while keeping the user in control. The proposed solution consists of the following key components:

a) Web Interface and User Management:

A responsive website with a home page, registration page, login page, dashboard, reply composer, history page and settings page. Users create an account, log in securely and manage their profile, signature and preferred reply tone.

b) Secure Mailbox Integration:

The user connects a mailbox using OAuth 2.0 authorisation, so that the system never sees the mailbox password. The mail module retrieves recent messages and conversation threads, removes quoted text and prepares clean content for analysis.

c) AI Reply Generation Engine:

The cleaned email, the conversation history, the user's signature and the selected tone are combined into a structured prompt. The prompt is sent to a Large Language Model, which returns a complete, context-aware reply. The prompt contains clear rules, such as not inventing dates, prices or commitments and writing in the language of the sender.

d) Tone and Style Control:

The user can choose formal, friendly, concise or detailed replies, and can regenerate a draft with a different tone or additional instruction, for example "decline politely" or "ask for the document again".

e) Human Review and Sending:

Every generated reply is shown in an editable text area. The user can edit, approve and send the reply from the website. No reply is sent without explicit approval, which prevents mistakes and builds trust.

f) History and Dashboard:

All generated and sent replies are stored with their status. The dashboard summarises the number of mails processed, replies generated, replies sent and the estimated time saved, helping users to understand the benefit of the system.

g) Security and Privacy:

Passwords are stored as salted hashes, sessions are protected with signed tokens, mailbox tokens are stored in encrypted form, and all communication takes place over HTTPS. Only the content needed for reply generation is sent to the AI service.

The proposed system combines intelligence, usability and security so that routine email can be handled quickly while important decisions remain with the user.

3.3 SOFTWARE COMPONENTS

SMART REPLY is implemented as a three-tier web application consisting of a browser-based front end, a Java back end and external services for mail and AI. The following software components are used in the implementation.

1. Front End (Website)

- **HTML5 and CSS3:** Used to build the structure and the responsive layout of the home page, login page, registration page, dashboard and reply composer.
- **JavaScript (ES6):** Handles form validation, calls to the REST API using the Fetch interface, dynamic updating of the mail list and display of the generated reply without reloading the page.

2. Back End (Server)

- **Java 17:** The main programming language of the server. The object-oriented design uses entities, repositories, services and controllers with clear responsibilities.
- **Spring Boot:** Provides the embedded web server, REST controllers, dependency injection and configuration management.
- **Spring Security and JWT:** Provide authentication, authorisation and token-based sessions.
- **Spring Data JPA (Hibernate):** Maps Java classes to database tables and performs database operations.

3. Database

- **MySQL:** Stores user accounts, preferences, connected mailbox details, generated replies and reply history. Password values are stored only as hashes and mailbox tokens only in encrypted form.

4. Mail Integration

- **Gmail API with OAuth 2.0:** Used to read messages and threads and to send approved replies on behalf of the user. IMAP and SMTP are supported as an alternative route for other providers.

5. AI Reply Engine

- **Large Language Model (LLM) API:** Receives the structured prompt over HTTPS and returns the generated reply text. The provider is configured through the application settings, so that the model can be changed without altering the code.
- **Prompt Builder:** A Java component that assembles the instruction, tone, user signature and conversation thread into a single prompt and limits the length to the allowed context size.

6. Security Utilities

- **BCrypt:** Hashes and verifies passwords [14].
- **JJWT library:** Creates and validates signed JSON Web Tokens [13].
- **AES encryption (javax.crypto):** Encrypts stored mailbox tokens.

7. Development and Testing Tools

- **Eclipse / IntelliJ IDEA and VS Code:** Development environments for Java and web files.
- **Maven:** Dependency management and project build.
- **Postman:** Testing of REST endpoints.
- **JUnit and Mockito:** Unit testing of services and controllers.
- **Git and GitHub:** Version control and team collaboration.

Layer	**Technology**	**Purpose**
Presentation	HTML5, CSS3, JavaScript	Website pages and user interaction
Application	Java 17, Spring Boot	REST API and business logic
Security	Spring Security, JWT, BCrypt, AES	Authentication and data protection
Data	MySQL, Spring Data JPA	Persistent storage of users and replies
Integration	Gmail API, OAuth 2.0, IMAP/SMTP	Reading mail and sending replies
Intelligence	LLM API, Prompt Builder	Generation of context-aware replies
Tools	Maven, Postman, JUnit, Git	Build, testing and version control

Table 3.1 Summary of Software Components

These components together provide a complete workflow, from user registration and mailbox connection, through AI reply generation and human review, to sending and tracking of replies, while keeping the system secure, maintainable and easy to use.

3.4 HARDWARE COMPONENTS

SMART REPLY is a software-oriented web application and does not require any specialised or embedded hardware. The application can be developed, tested and hosted on a standard computer or on a cloud server. Since the heavy computation of language generation is performed by the external AI service, the local hardware requirements are modest. Typical requirements are summarised in Table 3.2.

Component	**Development / Server**	**User (Client)**
Processor	Modern multi-core processor (Intel i5 or equivalent)	Any processor that can run a current web browser
Memory (RAM)	Minimum 8 GB	Minimum 2 GB
Storage	Minimum 10 GB free space for tools, database and logs	Not required
Network	Stable broadband connection for API calls	Internet connection
Display	Standard monitor	Desktop, laptop, tablet or mobile screen

Table 3.2 Hardware Requirements

3.5 USE CASES

i) Users of the System:

The primary users of the system are:

a) **Individual Users (Students, Professionals, Freelancers):** Connect their mailbox, receive AI-generated reply drafts, edit and send them quickly.

b) **Business and Support Teams:** Handle large numbers of customer or internal enquiries by generating consistent and polite replies to routine questions.

c) **Administrator:** Manages user accounts, monitors system usage, configures the AI provider and maintains the security of the platform.

ii) Use Cases:

a) **Individual User – Register and Log In**

Goal: Access the system securely. Description: The user opens the home page, creates an account with name, email and password, and logs in. The system validates the input, stores the password as a hash and issues a signed token for the session.

b) **Individual User – Connect Mailbox**

Goal: Allow the system to read incoming mail. Description: The user selects “Connect Mailbox” and is redirected to the mail provider to grant limited permission. After consent, the system stores the encrypted access token and displays the recent messages on the dashboard.

c) **Individual User – Generate Reply**

Goal: Obtain a ready-to-send reply. Description: The user opens an email, selects a tone and clicks “Generate Reply”. The system builds the prompt, calls the AI engine and displays the draft in the composer within a few seconds.

d) **Individual User – Review, Edit and Send**

Goal: Send a correct reply. Description: The user reads the draft, edits it if necessary, regenerates it with a different instruction or approves it. The system sends the mail through the connected account and stores it in the history.

e) **Administrator – Monitor and Configure**

Goal: Keep the platform reliable. Description: The administrator reviews usage statistics, manages accounts and updates the AI configuration and rate limits.

iii) Usage Scenarios:

a) **Quick Response to a Routine Query**

User: Student. Situation: A professor asks for confirmation of a submission date. Action: The student selects a formal tone and generates a reply. Outcome: A polite confirmation is ready in seconds and the student sends it after a quick check.

b) **Customer Enquiry Handling**

User: Support executive. Situation: Twenty customers ask about delivery status. Action: The executive generates replies one after another, adding the order status as an instruction. Outcome: Consistent and courteous replies are sent in a fraction of the usual time.

c) **Declining a Request Politely**

User: Working professional. Situation: A meeting request clashes with another commitment. Action: The user adds the instruction “decline politely and suggest next week”. Outcome: A well-worded refusal with an alternative time is generated.

d) **Communication in a Different Tone**

User: Freelancer. Situation: A friendly client message needs a warm reply, while a payment reminder needs a formal one. Action: The freelancer switches the tone for each message. Outcome: Every reply matches the relationship and the purpose of the mail.

CHAPTER 4

IMPLEMENTATION

The implementation of the SMART REPLY project began with the design of the website pages and the database. The home page, registration page, login page, dashboard, reply composer, history page and settings page were designed with HTML, CSS and JavaScript, and a REST API was created using Java and Spring Boot to serve these pages. User accounts were secured with salted password hashing and signed session tokens. The mail integration module was then developed to connect to the user's mailbox through OAuth 2.0 and to retrieve and clean incoming messages. For intelligence, an AI reply generation module was implemented in which the cleaned message, the conversation history, the user's signature and the chosen tone are assembled into a structured prompt and sent to a Large Language Model. The returned draft is displayed in an editable composer so that the user can review, modify and approve it before sending. Finally, a dashboard module was added to present usage statistics, reply status and history, so that users can see how much time the system saves.

4.1 EXPERIMENTAL SETUP

The experimental setup for SMART REPLY was designed to provide a reliable environment for development, testing and demonstration. The application was developed on a system with an Intel i5 processor, 8 GB RAM and the Windows 10 operating system. The back end was written in Java 17 using the Spring Boot framework and built with Maven, while the front end used HTML5, CSS3 and JavaScript. MySQL was used as the database and the project was developed in IntelliJ IDEA and Visual Studio Code. REST endpoints were tested with Postman, and the web pages were tested in current versions of Google Chrome and Microsoft Edge.

A test mailbox was created for the experiments so that no real personal mail was used. A set of sample emails covering different situations, such as meeting requests, document requests, complaints, thank-you messages and questions with several parts, was prepared and sent to this mailbox. The mailbox was connected to the application through OAuth 2.0 with limited permission to read and send mail. The AI provider details and encryption keys were stored in environment variables rather than in the source code. For each sample mail, replies were generated in different tones and examined for relevance, correctness of facts, politeness

and length.

The user interface was tested at desktop, tablet and mobile widths to check the responsive behaviour. Unit tests were written for the services that build prompts, clean mail content and validate user input, and integration tests were used for the authentication and reply endpoints. The final application was run in a local environment, and it can be deployed on a cloud server for real-world use.

4.2 SYSTEM ARCHITECTURE

The architecture of SMART REPLY follows a layered client-server design, shown in Figure 4.1. The browser-based client communicates with a Spring Boot server through REST endpoints over HTTPS. The server is organised into controllers, services and repositories, and it connects to the database, the mail provider and the AI service.

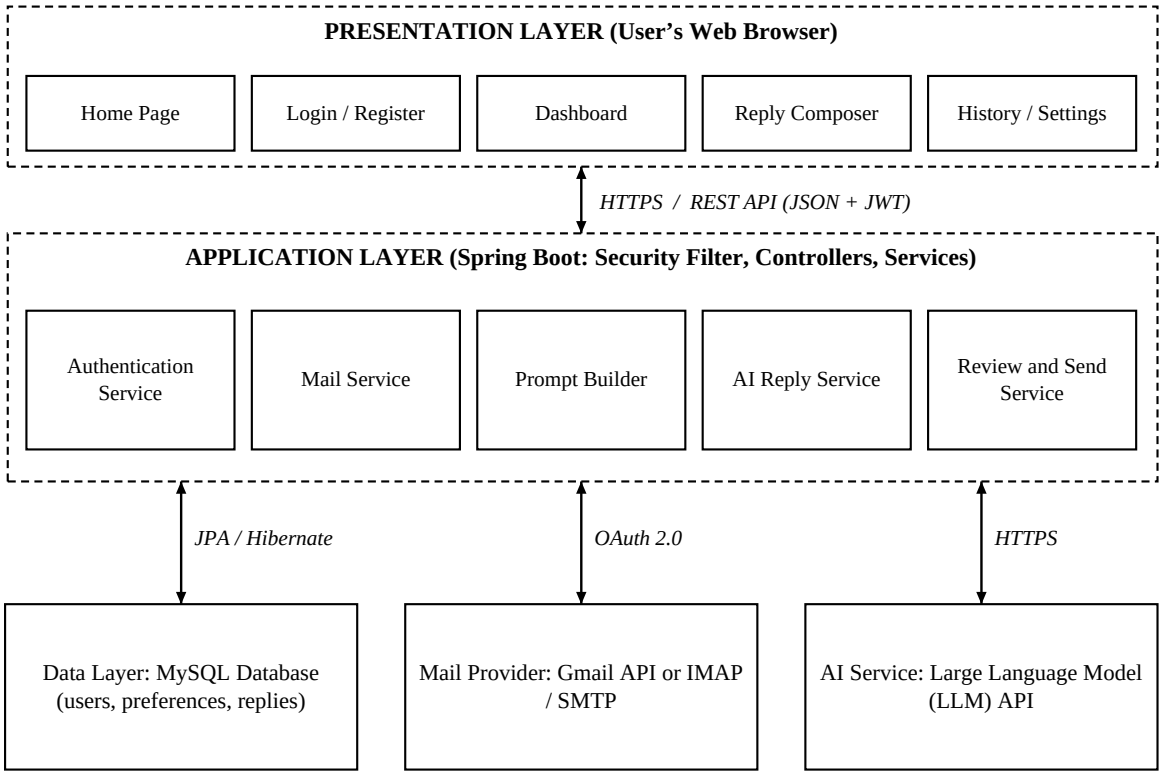


Figure 4.1 System Architecture of SMART REPLY

The main layers of the architecture are described below.

- **Presentation Layer:** The website pages (home, login, registration, dashboard, composer, history and settings) rendered in the user's browser.

- **API and Security Layer:** REST controllers protected by a JWT filter. Every request except registration, login and public pages must carry a valid token.
- **Service Layer:** Business logic implemented in the Authentication Service, Mail Service, Prompt Builder, AI Reply Service and Review Service.
- **Data Layer:** JPA repositories that read and write users, mailbox connections, reply drafts and history in the MySQL database.
- **External Services:** The mail provider (Gmail API or IMAP/SMTP) for reading and sending mail, and the LLM API for generating reply text.

The workflow of generating a reply is shown in Figure 4.2. When the user selects a message, the server retrieves and cleans the content, checks whether a reply is appropriate, builds the prompt, calls the AI service and returns the draft. The user then edits or approves the draft, and only after approval is the reply sent and recorded in the history.

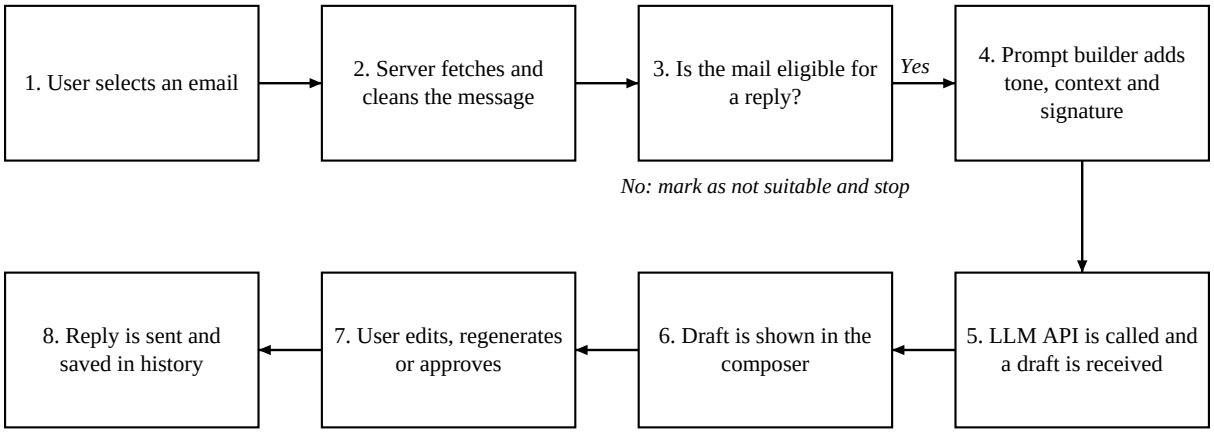


Figure 4.2 Reply Generation Workflow

The database consists of four main tables, summarised in Table 4.1, and the main REST endpoints exposed by the server are listed in Table 4.2.

Table	**Important Columns**	**Description**
users	id, name, email, password_hash, created_at	Registered users with hashed passwords
mail_accounts	id, user_id, provider, email, enc_token, expiry	Connected mailboxes with encrypted access tokens

Table	Important Columns	Description
preferences	id, user_id, tone, signature, language	Default tone, signature and language for each user
replies	id, user_id, message_id, draft, final_text, status, created_at	Generated drafts and final replies with status (DRAFT, SENT, DISCARDED)

Table 4.1 Database Tables

Method and Endpoint	Purpose
POST /api/auth/register	Create a new user account
POST /api/auth/login	Verify credentials and return a JWT
GET /api/mail/connect	Start OAuth 2.0 mailbox connection
GET /api/mail/messages	List recent messages of the connected mailbox
GET /api/mail/messages/{id}	Retrieve a single message with its thread
POST /api/reply/generate	Generate an AI reply for a message and tone
POST /api/reply/send	Send the approved reply and save it in history
GET /api/reply/history	List previous replies of the user
GET and PUT /api/settings	Read and update tone, signature and language
GET /api/dashboard/stats	Return usage statistics for the dashboard

Table 4.2 Main REST API Endpoints

4.3 MODULE IMPLEMENTATION DETAILS

This section details the step-by-step implementation of each of the five primary modules of SMART REPLY: the User Authentication Module, the Mail Integration Module, the AI Reply Generation Module, the Reply Review and Send Module, and the Web Interface and Dashboard Module.

4.3.1 User Authentication Module

4.3.1.1 Algorithm Used

The User Authentication Module is responsible for registering users, verifying their identity and protecting every private page and API call. The algorithm combines salted password hashing with token-based sessions:

1. **Register:** Receive name, email and password; validate the format; reject the request if the email already exists.
2. **Hash:** Convert the password into a salted BCrypt hash and store only the hash in the database.
3. **Login:** Find the user by email and compare the entered password with the stored hash.
4. **Issue Token:** On success, create a signed JSON Web Token containing the user identity and an expiry time.
5. **Verify:** For every later request, a filter validates the token signature and expiry before the request reaches the controller.

4.3.1.2 Mathematical Implementation

The mathematical part of this module is limited to token signing and password hashing cost.

- Token signature (HMAC with SHA-256):

$$Signature = \mathrm{HMAC}_{\mathrm{SHA256}}(K, \mathrm{Base64Url}(Header) + \text{“.”} + \mathrm{Base64Url}(Payload)) \qquad (4.1)$$

where K is the secret key known only to the server. A token is accepted only if the recomputed signature matches the received one.

- BCrypt work factor:

$$Iterations = 2^{cost} \quad (4.2)$$

where *cost* is the work factor (10 in this project), which makes each password check deliberately slow and therefore resistant to brute-force attacks [14].

- Token validity:

$$t_{now} < t_{exp}, \quad t_{exp} = t_{issued} + 3600 \text{ s} \quad (4.3)$$

A token is valid only until its expiry time, which is one hour after it was issued.

4.3.1.3 Experimental Setup

- **Test Accounts:** Several test users with valid and invalid inputs (empty fields, wrong email format, short password, duplicate email).
- **Scenarios Tested:** Successful registration, login with correct and wrong password, access to a private endpoint with no token, with a tampered token and with an expired token.
- **Tools:** Postman for API calls and JUnit for unit tests of the service.

4.3.1.4 Functional Components of Coding

The module consists of a registration method, a login method and a token utility:

```
@PostMapping("/register")
public ResponseEntity<String> register(@Valid @RequestBody RegisterRequest r) {
    if (userRepo.existsByEmail(r.getEmail()))
        return ResponseEntity.status(HttpStatus.CONFLICT)
                             .body("Email already registered");
    User u = new User(r.getName(), r.getEmail(),
                      encoder.encode(r.getPassword()));
    userRepo.save(u);
    return ResponseEntity.ok("Registration successful");
}

public String generateToken(String email) {
    return Jwts.builder().setSubject(email)
        .setIssuedAt(new Date())
        .setExpiration(new Date(System.currentTimeMillis() + 3600_000))
        .signWith(key, SignatureAlgorithm.HS256).compact();
}
```

4.3.1.5 Implementation Using Tools

- **Software:** Java 17, Spring Boot, Spring Security, JJWT, BCrypt, MySQL.
- **Hardware:** Standard laptop or desktop; no additional hardware is required.

4.3.2 Mail Integration Module

4.3.2.1 Algorithm Used

The Mail Integration Module connects the application to the user's mailbox and prepares the messages for analysis. It follows an Authorise–Fetch–Clean pipeline:

1. **Authorise:** Redirect the user to the mail provider, obtain consent through OAuth 2.0 and receive an access token and a refresh token.
2. **Store:** Encrypt the tokens with AES and save them against the user account.
3. **Fetch:** Retrieve the list of recent messages and, for a selected message, its complete thread.
4. **Clean:** Decode the message body, remove HTML tags, quoted earlier replies, signatures and disclaimers, and keep the latest message and a short thread summary.
5. **Filter:** Mark automated messages such as "no-reply" notifications and newsletters as not suitable for an automatic reply.

4.3.2.2 Mathematical Implementation

- Eligibility of a message for reply generation:

$$Eligible(m) = \text{NOT } NoReply(m) \text{ AND NOT } Bulk(m) \text{ AND } (m \text{ not yet replied}) \quad (4.4)$$

where *NoReply* and *Bulk* are boolean checks based on the sender address and the mail headers.

- Retry delay with exponential back-off when the mail service is temporarily unavailable:

$$d_k = d_0 \cdot 2^k \quad (4.5)$$

where d_0 is the initial delay (1 second), k is the retry number and the number of retries is limited to three.

4.3.2.3 Experimental Setup

- **Mailbox:** A dedicated test mailbox connected through OAuth 2.0 with read and send permissions only.

- **Sample Mail:** Plain text mails, HTML mails, long threads with quoted replies, mails with attachments and automated notifications.
- **Checks:** Correct decoding of the body, removal of quoted text and signatures, and correct detection of automated mail.

4.3.2.4 Functional Components of Coding

```
public List<MailDto> fetchRecent(User user, int max) {
    String token = crypto.decrypt(user.getMailAccount().getEncToken());
    String url = "https://gmail.googleapis.com/gmail/v1/users/me/messages"
               + "?maxResults=" + max + "&q=in:inbox";
    HttpHeaders h = new HttpHeaders();
    h.setBearerAuth(token);
    ResponseEntity<GmailList> res = rest.exchange(url, HttpMethod.GET,
                                    new HttpEntity<>(h), GmailList.class);
    return res.getBody().getMessages().stream()
              .map(m -> getMessage(token, m.getId()))
              .filter(this::isEligible)
              .collect(Collectors.toList());
}
```

4.3.2.5 Implementation Using Tools

- **Software:** Gmail API, OAuth 2.0, Spring RestTemplate, Jsoup for cleaning HTML content.
- **Hardware:** Standard laptop; internet connection required.

4.3.3 AI Reply Generation Module

4.3.3.1 Algorithm Used

The AI Reply Generation Module is the core of the system. It converts a cleaned email into a structured prompt, calls a Large Language Model and post-processes the returned text. The steps of the algorithm are:

1. **Collect Context:** Take the latest message, a short summary of earlier messages in the thread, the sender name and the subject.
2. **Load Preferences:** Read the user's tone, language and signature, and any additional instruction typed for this reply.
3. **Build Prompt:** Combine a fixed system instruction (rules such as "do not invent facts") with the context and preferences.

4. **Limit Length:** Truncate the oldest part of the thread if the prompt exceeds the allowed number of tokens.
5. **Generate:** Send the prompt to the LLM API with a timeout and retry policy.
6. **Post-process:** Remove unwanted text, add the signature, and return the draft with a status of DRAFT.

4.3.3.2 Mathematical Implementation

- Prompt construction:

$$Prompt = S \,||\, U \,||\, T \,||\, C \tag{4.6}$$

where S is the system instruction, U the user instruction, T the tone and C the email context, and || denotes concatenation. The total length must satisfy $|Prompt| \le L_{\max}$ tokens.

- Autoregressive generation of the reply y by the language model [6][8]:

$$P(y \mid x) = \prod_{t=1}^{n} P(y_t \mid y_{<t}, x) \tag{4.7}$$

where x is the prompt and y_t is the word (token) generated at step t.

- Temperature-controlled selection of the next token:

$$p_i = \exp(z_i / \tau) \,/\, \sum_j \exp(z_j / \tau) \tag{4.8}$$

where z_i is the score of token i and τ is the temperature. A low value of τ (for example 0.3) produces stable and formal replies, whereas a higher value produces more varied wording.

4.3.3.3 Experimental Setup

- **Sample Emails:** Meeting requests, document requests, complaints, thank-you notes, invitations and multi-question mails.
- **Tones Tested:** Formal, friendly, concise and detailed.
- **Evaluation Criteria:** Relevance to the question asked, absence of invented facts, correct tone, grammar and suitable length.
- **Settings:** Low temperature for formal tone, moderate temperature for friendly tone, and a timeout of 20 seconds per request.

The tone instructions used in the prompt are summarised in Table 4.3.

Tone	Instruction added to the prompt	Temperature
Formal	Use polite, professional language, full sentences and a respectful closing.	0.3
Friendly	Use a warm, natural and positive style while remaining respectful.	0.6
Concise	Answer in at most three short sentences and avoid repetition.	0.3
Detailed	Address every question in the mail point by point with clear explanation.	0.4

Table 4.3 Tone Options and Prompt Instructions

4.3.3.4 Functional Components of Coding

```
public String buildPrompt(MailDto mail, Preference p, String extra) {
    return "You are an email assistant. Write a reply below.\n"
         + "Rules: do not invent dates, prices or promises; reply in the "
         + "language of the sender; keep the tone " + p.getTone() + ".\n"
         + (extra == null ? "" : "Extra instruction: " + extra + "\n")
         + "Subject: " + mail.getSubject() + "\n"
         + "From: " + mail.getFromName() + "\n"
         + "Thread summary: " + mail.getThreadSummary() + "\n"
         + "Latest message:\n" + mail.getCleanBody();
}

public String generate(MailDto mail, Preference p, String extra) {
    String prompt = buildPrompt(mail, p, extra);
    String text = llmClient.complete(prompt, p.getTemperature());
    return text.trim() + "\n\n" + p.getSignature();
}
```

4.3.3.5 Implementation Using Tools

- **Software:** Java, Spring Boot, HTTP client for the LLM API, JSON processing with Jackson.
- **Hardware:** Standard laptop or cloud server; the language model itself runs on the provider’s infrastructure.

4.3.4 Reply Review and Send Module

4.3.4.1 Algorithm Used

The Reply Review and Send Module keeps the user in control of every message. The generated draft is displayed in an editable area, and the following workflow is used:

1. **Display:** Show the original message on one side and the draft reply on the other.
2. **Edit or Regenerate:** The user edits the text manually or asks for a new draft with a different tone or instruction.

3. **Approve:** The user clicks “Send”, which is the only action that triggers sending.
4. **Send:** The server builds a standard reply message with the correct recipient, subject and thread reference and sends it through the mail provider.
5. **Record:** The final text and its status (SENT or DISCARDED) are stored in the history table.

4.3.4.2 Mathematical Implementation

- Acceptance rate of generated replies:

$$AcceptanceRate = (Replies\ sent\ without\ change / Total\ replies\ generated) \times 100 \quad (4.9)$$

- Edit ratio between the generated draft R_g and the final reply R_f using edit distance d:

$$EditRatio = d(R_g, R_f) / \max(|R_g|, |R_f|) \quad (4.10)$$

A low edit ratio indicates that the draft needed little correction and that the generated replies are of good quality.

4.3.4.3 Experimental Setup

- **Scenarios:** Send without editing, send after editing, regenerate and then send, and discard a draft.
- **Checks:** Correct recipient and subject, correct placement in the original thread, and correct recording of status in the history.

4.3.4.4 Functional Components of Coding

```
@PostMapping("/send")
public ResponseEntity<String> send(@RequestBody SendRequest req, Principal pr) {
    User u = userService.byEmail(pr.getName());
    Reply r = replyRepo.findByIdAndUserId(req.getReplyId(), u.getId())
                       .orElseThrow(() -> new NotFoundException("Reply"));
    mailService.sendReply(u, r.getMessageId(), req.getFinalText());
    r.setFinalText(req.getFinalText());
    r.setStatus(Status.SENT);
    replyRepo.save(r);
    return ResponseEntity.ok("Reply sent");
}
```

4.3.4.5 Implementation Using Tools

- **Software:** Spring Boot, Gmail API (send), Spring Data JPA, MySQL.
- **Hardware:** Standard laptop; internet connection required.

4.3.5 Web Interface and Dashboard Module

4.3.5.1 Algorithm Used

- The website is built with HTML5, CSS3 and JavaScript and communicates with the server through the Fetch interface using JSON.
- Pages include the home page, registration page, login page, dashboard, reply composer, history page and settings page.
- The dashboard presents the number of mails processed, replies generated and sent, and the estimated time saved.

4.3.5.2 Mathematical Implementation

This module relies mainly on aggregation functions:

$$TimeSaved = (T_{\text{manual}} - T_{\text{assisted}}) / T_{\text{manual}} \times 100 \tag{4.11}$$

where T_{manual} is the average time needed to write a reply manually and T_{assisted} is the average time needed with SMART REPLY, including reading and editing.

$$AvgResponseTime = (1 / N) \sum_{i=1}^{N} (t_{\text{reply},i} - t_{\text{received},i}) \tag{4.12}$$

4.3.5.3 Experimental Setup

- Website tested in Google Chrome and Microsoft Edge at desktop, tablet and mobile widths.
- Components: forms, mail list, reply composer, tone selector, history table and statistic cards.
- Users: individual users, support teams and administrators.

4.3.5.4 Functional Components of Coding

```
async function generateReply(messageId, tone) {
  const res = await fetch("/api/reply/generate", {
    method: "POST",
    headers: { "Content-Type": "application/json",
               "Authorization": "Bearer " + localStorage.getItem("token") },
    body: JSON.stringify({ messageId: messageId, tone: tone })
  });
  if (!res.ok) { showError("Could not generate reply"); return; }
  const data = await res.json();
  document.getElementById("draft").value = data.draft;
}
```

4.3.5.5 Implementation Using Tools

- **Software:** HTML5, CSS3, JavaScript (ES6), Spring Boot static resources.
- **Hardware:** Standard laptop or desktop; the site can be hosted on a cloud server for real-world use.

4.4 OUTPUT AND SCREENSHOTS

This section presents the main pages of the SMART REPLY website and explains the output that the user sees on each page.

4.4.1 Home Page

The home page is the first page that a visitor sees. It introduces SMART REPLY with a short heading, a description of how the service works in three simple steps (connect mailbox, generate reply, review and send), a summary of the main features and clear “Login” and “Get Started” buttons in the navigation bar.

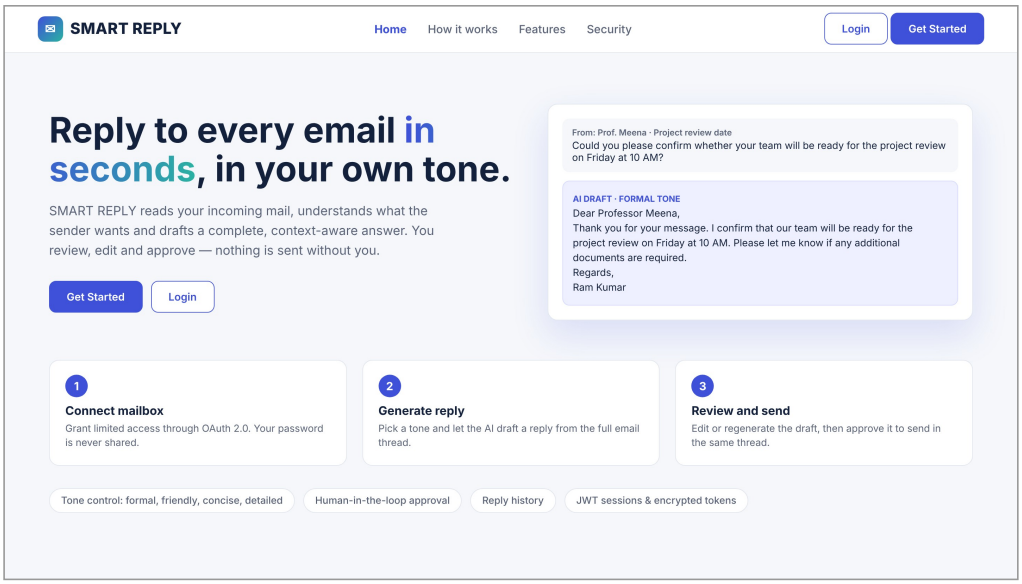


Figure 4.3 Home Page

4.4.2 Registration and Login Pages

The registration page collects the name, email address and password of a new user, and shows validation messages for incorrect input. The login page accepts the email and password and, after successful verification, redirects the user to the dashboard. A link on each page allows the user to move to the other page.

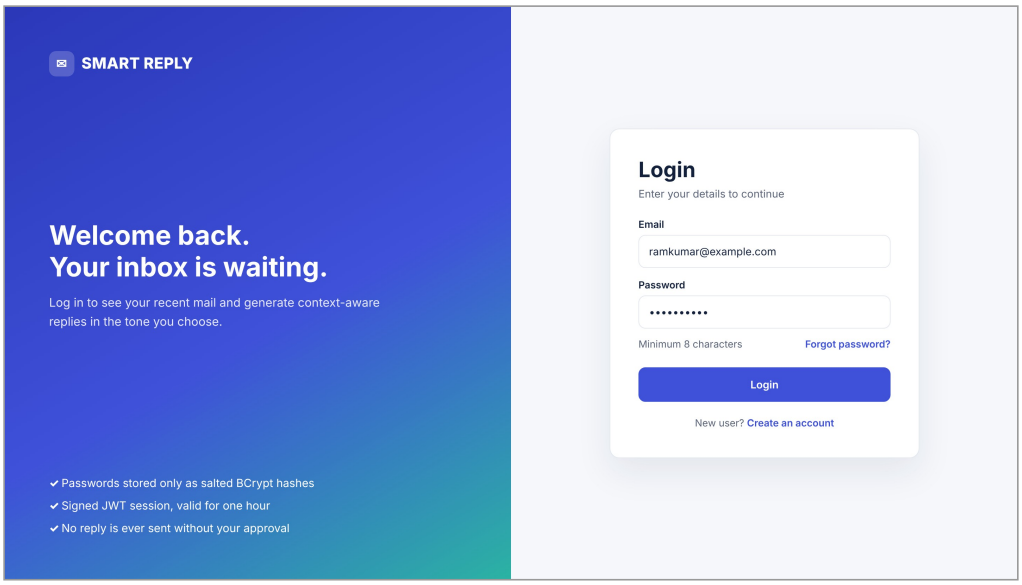


Figure 4.4 Login Page

4.4.3 Dashboard and Inbox View

After login, the dashboard displays the connected mailbox with the list of recent messages, showing the sender, subject and time. Statistic cards at the top present the number of messages processed, replies generated, replies sent and the estimated time saved. Messages that are not suitable for an automatic reply, such as newsletters, are marked accordingly.

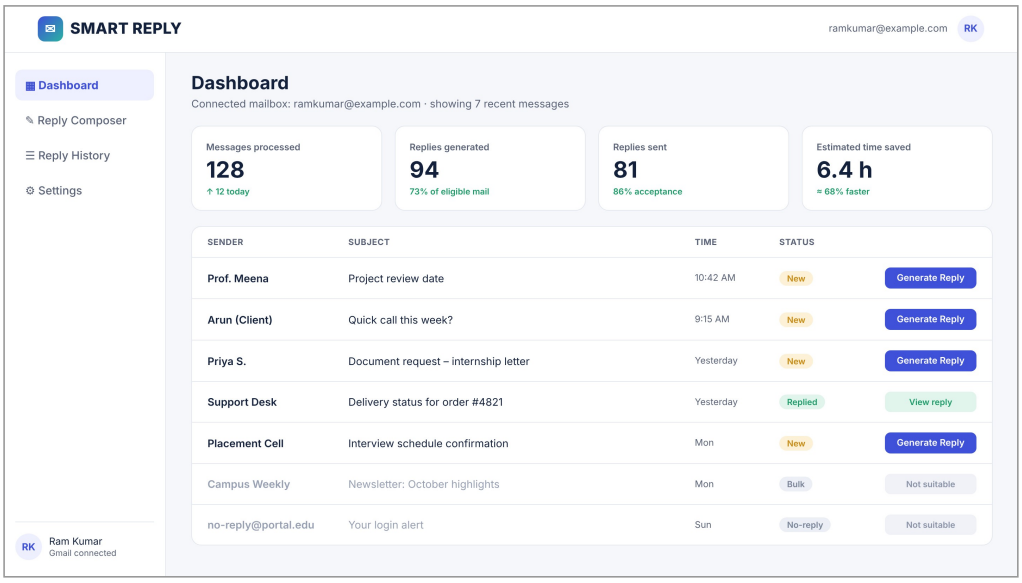


Figure 4.5 Dashboard and Inbox View

4.4.4 AI Reply Composer

When a message is selected, the composer opens with the original email on one side and the AI-generated draft on the other. The user can choose the tone, type an additional instruction, regenerate the draft, edit it manually and press “Send”. A loading indicator is shown while the reply is being generated, and an error message appears if the AI service is not reachable.

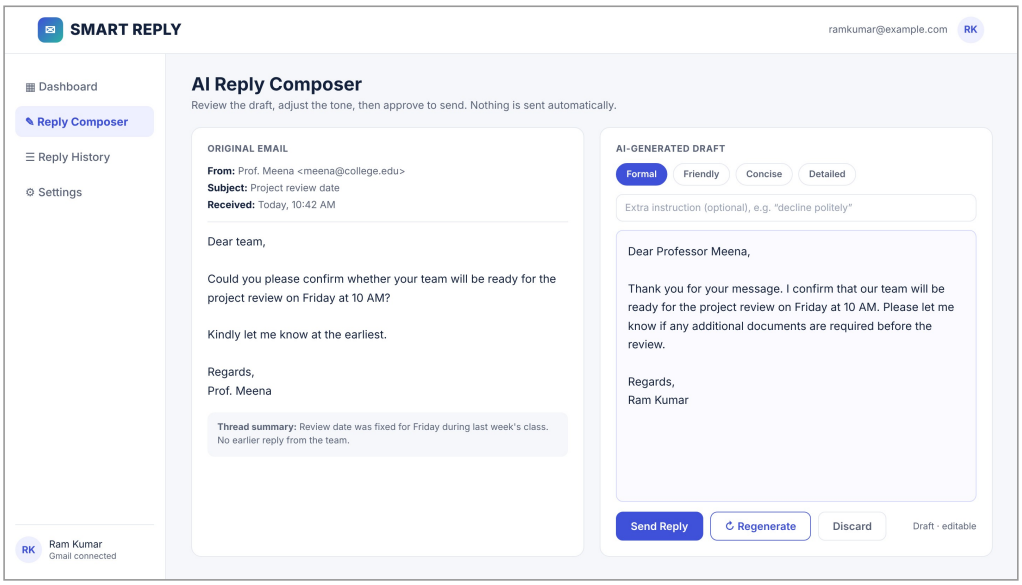


Figure 4.6 AI Reply Composer with Generated Draft

4.4.5 Reply History and Settings

The history page lists previously generated and sent replies with their status, date and the original subject, and allows a reply to be opened for reference. The settings page lets the user change the default tone, the signature, the language and the connected mailbox.

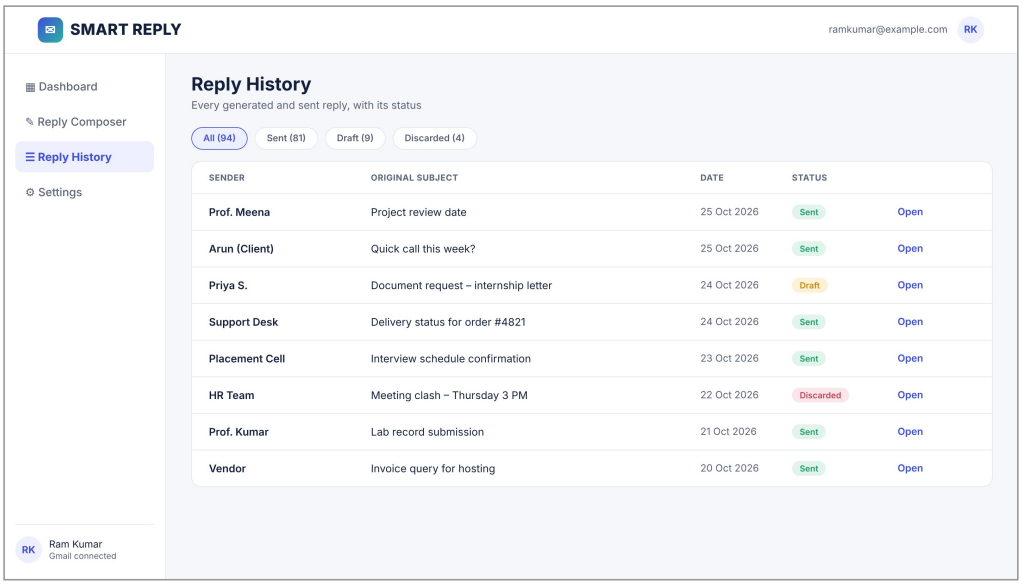


Figure 4.7 Reply History Page

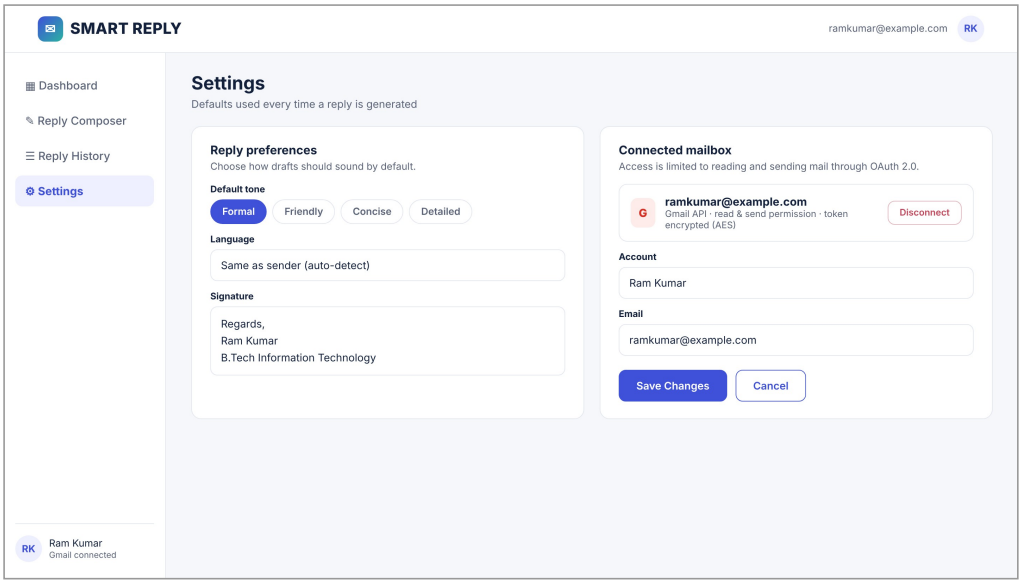


Figure 4.8 Settings Page

4.4.6 Test Cases

To verify the behaviour of the system, a set of test cases was designed. Table 4.4 lists the test cases and the expected behaviour of the system for each case.

No.	Test Case	Expected Behaviour
1	Register with a new email and valid password	Account is created and the password is stored only as a hash
2	Register with an existing email	Request is rejected with a clear message

No.	Test Case	Expected Behaviour
3	Login with a wrong password	Access is denied and no token is issued
4	Open the dashboard without a token	Request is rejected and the user is asked to log in
5	Open the dashboard with an expired token	Request is rejected and the user is asked to log in again
6	Connect the mailbox and open the inbox	Recent messages are listed with sender, subject and time
7	Generate a formal reply to a meeting request	A polite reply that addresses the request is displayed in the composer
8	Generate a reply for a no-reply notification	System marks the message as not suitable for automatic reply
9	Regenerate with the instruction “decline politely”	A new draft that politely declines is displayed
10	Send an approved reply	Reply is sent in the same thread and saved as SENT in history
11	AI service unavailable	A friendly error is shown and the user can retry

Table 4.4 Test Cases and Expected Behaviour

CHAPTER 5

CONCLUSION AND FUTURE ENHANCEMENTS

5.1 CONCLUSION

The project aimed to develop SMART REPLY, an AI-powered automatic email reply website that assists users in responding to their mail quickly, accurately and in a suitable tone. The work undertaken has met this objective through the systematic design and integration of a web interface, a secure Java back end, mailbox integration and a Large Language Model based reply engine.

The project began with the design of the website, including the home page, registration page, login page, dashboard, composer, history page and settings page. A secure authentication module using salted password hashing and signed tokens was implemented so that every private page and API call is protected. This ensured that the system could be used safely by many users with separate accounts and mailboxes.

Next, the mail integration module was developed to connect to the user's mailbox through OAuth 2.0, to retrieve messages and threads and to clean them by removing quoted text, signatures and markup. Automated and bulk messages are detected and excluded from reply generation. The cleaned content became the reliable input on which the AI module operates.

The AI reply generation module forms the central contribution of the project. By combining a carefully written system instruction, the conversation context, the user's signature and the selected tone into a structured prompt, the module produces complete and context-aware replies instead of short fixed phrases. The tone options and the regeneration facility give the user control over the style of the reply.

A significant feature of the project is the human-in-the-loop design. No reply is sent without the explicit approval of the user, who can edit or regenerate the draft. This protects the user from incorrect or inappropriate messages and makes the system trustworthy for professional communication. The dashboard and history modules complete the system by presenting usage statistics and a record of all replies, so that the benefit of the system can be understood clearly.

Key Achievements of the Project:

- Design and development of a responsive website with home, login, registration, dashboard, composer, history and settings pages.
- Implementation of secure authentication using BCrypt password hashing and JWT-based sessions.
- Secure mailbox integration through OAuth 2.0 with cleaning and filtering of incoming mail.
- Development of an AI reply engine that produces context-aware replies in a selectable tone.
- Introduction of a review, edit and approve workflow so that the user always remains in control.
- Creation of a dashboard and reply history that present usage statistics and track every reply.
- Fulfilment of the primary objective: reducing the time and effort spent on routine email while maintaining quality and security.

5.2 FUTURE ENHANCEMENTS

While the project successfully addressed its objectives, there remains significant scope for extension and improvement. Some possible directions include:

a) Adoption of Advanced AI Techniques

- Using retrieval-augmented generation so that replies can draw on the user's own documents, FAQs and previous replies [10].
- Learning the personal writing style of each user from the history of edited replies, so that drafts need fewer corrections.
- Adding automatic classification of mail by intent and priority, so that urgent messages are shown first.

b) Support for More Mail Providers and Languages

- Extending the integration to Outlook, Yahoo and other providers using IMAP, SMTP and their own APIs.
- Providing multilingual replies and automatic translation of incoming and outgoing messages.

c) Optional Automation with Safeguards

- Allowing fully automatic replies for pre-approved, low-risk categories of mail, such as acknowledgement of receipt, with rules, schedules and an easy way to switch off.
- Adding scheduled sending, reminders for unanswered mail and follow-up suggestions.

d) Enhancement of the Dashboard and Analytics

- Adding charts for response time, reply volume and quality trends over weeks and months.
- Providing team dashboards and shared reply templates for support teams.

e) Scalability, Privacy and Deployment

- Deploying the system as a cloud-based service with load balancing, caching and monitoring.
- Offering the choice of a privately hosted language model for organisations that cannot send mail content to an external service.
- Developing mobile applications and a browser extension so that replies can be generated directly inside the mail client.

APPENDIX I

PROJECT STRUCTURE

```
smartreply/
 |-- pom.xml
 |-- src/main/java/com/smartreply/
 |    |-- SmartReplyApplication.java
 |    |-- config/SecurityConfig.java
 |    |-- security/JwtUtil.java, JwtFilter.java
 |    |-- model/User.java, MailAccount.java, Preference.java, Reply.java
 |    |-- repository/UserRepository.java, ReplyRepository.java
 |    |-- service/AuthService.java, MailService.java,
 |    |           PromptBuilder.java, ReplyService.java, CryptoService.java
 |    |-- controller/AuthController.java, MailController.java,
 |                   ReplyController.java, SettingsController.java
 |-- src/main/resources/
      |-- application.properties
      |-- static/index.html, login.html, register.html,
                 dashboard.html, composer.html, history.html,
                 settings.html, css/style.css, js/app.js
```

APPLICATION CONFIGURATION

```
spring.datasource.url=jdbc:mysql://localhost:3306/smartreply
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASS}
spring.jpa.hibernate.ddl-auto=update

app.jwt.secret=${JWT_SECRET}
app.jwt.expiry-ms=3600000
app.crypto.key=${TOKEN_ENC_KEY}

app.llm.endpoint=${LLM_ENDPOINT}
app.llm.api-key=${LLM_API_KEY}
app.llm.timeout-seconds=20
```

USER ENTITY

```
@Entity
@Table(name = "users")
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(name = "password_hash", nullable = false)
    private String passwordHash;

    private LocalDateTime createdAt = LocalDateTime.now();

    public User() { }
    public User(String name, String email, String passwordHash) {
```

```
        this.name = name;
        this.email = email;
        this.passwordHash = passwordHash;
    }
    // getters and setters
}
```

JWT UTILITY AND FILTER

```
@Component
public class JwtUtil {
    private final Key key;
    private final long expiryMs;

    public JwtUtil(@Value("${app.jwt.secret}") String secret,
                   @Value("${app.jwt.expiry-ms}") long expiryMs) {
        this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
        this.expiryMs = expiryMs;
    }

    public String generateToken(String email) {
        Date now = new Date();
        return Jwts.builder().setSubject(email).setIssuedAt(now)
                   .setExpiration(new Date(now.getTime() + expiryMs))
                   .signWith(key, SignatureAlgorithm.HS256).compact();
    }

    public String validateAndGetEmail(String token) {
        return Jwts.parserBuilder().setSigningKey(key).build()
                   .parseClaimsJws(token).getBody().getSubject();
    }
}

public class JwtFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest req,
            HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            try {
                String email = jwtUtil.validateAndGetEmail(header.substring(7));
                var auth = new UsernamePasswordAuthenticationToken(
                        email, null, List.of());
                SecurityContextHolder.getContext().setAuthentication(auth);
            } catch (JwtException e) {
                res.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
                return;
            }
        }
        chain.doFilter(req, res);
    }
}
```

AUTHENTICATION CONTROLLER

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {
    @Autowired private UserRepository userRepo;
    @Autowired private PasswordEncoder encoder;
    @Autowired private JwtUtil jwtUtil;

    @PostMapping("/register")
    public ResponseEntity<String> register(@Valid @RequestBody RegisterRequest r) {
        if (userRepo.existsByEmail(r.getEmail()))
            return ResponseEntity.status(HttpStatus.CONFLICT)
                                 .body("Email already registered");
        userRepo.save(new User(r.getName(), r.getEmail(),
                               encoder.encode(r.getPassword())));
        return ResponseEntity.ok("Registration successful");
    }

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequest r) {
        Optional<User> u = userRepo.findByEmail(r.getEmail());
        if (u.isEmpty() ||
            !encoder.matches(r.getPassword(), u.get().getPasswordHash()))
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
                                 .body("Invalid email or password");
        return ResponseEntity.ok(
            Map.of("token", jwtUtil.generateToken(u.get().getEmail())));
    }
}
```

MAIL SERVICE (FETCH AND CLEAN)

```
@Service
public class MailService {
    @Autowired private RestTemplate rest;
    @Autowired private CryptoService crypto;

    public List<MailDto> fetchRecent(User user, int max) {
        String token = crypto.decrypt(user.getMailAccount().getEncToken());
        String url = "https://gmail.googleapis.com/gmail/v1/users/me/messages"
                   + "?maxResults=" + max + "&q=in:inbox";
        ResponseEntity<GmailList> res = rest.exchange(url, HttpMethod.GET,
                new HttpEntity<>(bearer(token)), GmailList.class);
        List<MailDto> out = new ArrayList<>();
        for (GmailRef ref : res.getBody().getMessages()) {
            MailDto m = getMessage(token, ref.getId());
            if (isEligible(m)) out.add(m);
        }
        return out;
    }

    public String cleanBody(String html) {
        String text = Jsoup.parse(html).text();
        int cut = text.indexOf(" wrote:");          // remove quoted history
        if (cut > 0) text = text.substring(0, cut);
        return text.replaceAll("\\s+", " ").trim();
    }

    private boolean isEligible(MailDto m) {
```

```
        String from = m.getFromEmail().toLowerCase();
        return !from.contains("no-reply") && !from.contains("noreply")
            && !m.hasHeader("List-Unsubscribe");
    }

    public void sendReply(User user, String messageId, String body) {
        // builds a MIME message, base64url-encodes it and calls
        // POST /gmail/v1/users/me/messages/send with the thread id
    }
}
```

PROMPT BUILDER AND REPLY SERVICE

```
@Service
public class ReplyService {
    @Autowired private LlmClient llmClient;
    @Autowired private ReplyRepository replyRepo;

    public Reply generate(User user, MailDto mail, Preference p, String extra) {
        String prompt = buildPrompt(mail, p, extra);
        String text = llmClient.complete(prompt, temperatureFor(p.getTone()));
        Reply r = new Reply();
        r.setUserId(user.getId());
        r.setMessageId(mail.getId());
        r.setDraft(text.trim() + "\n\n" + p.getSignature());
        r.setStatus(Status.DRAFT);
        return replyRepo.save(r);
    }

    String buildPrompt(MailDto mail, Preference p, String extra) {
        StringBuilder sb = new StringBuilder();
        sb.append("You are an email assistant. Write a reply to this email.\n");
        sb.append("Rules: do not invent dates, prices or promises; reply in the ");
        sb.append("language of the sender; keep the tone ").append(p.getTone());
        sb.append(".\n");
        if (extra != null && !extra.isBlank())
            sb.append("Extra instruction: ").append(extra).append("\n");
        sb.append("Subject: ").append(mail.getSubject()).append("\n");
        sb.append("From: ").append(mail.getFromName()).append("\n");
        sb.append("Thread summary: ").append(mail.getThreadSummary()).append("\n");
        sb.append("Latest message:\n").append(mail.getCleanBody());
        return sb.toString();
    }

    double temperatureFor(String tone) {
        return switch (tone) {
            case "friendly" -> 0.6;
            case "detailed" -> 0.4;
            default -> 0.3;
        };
    }
}
```

REPLY CONTROLLER

```
@RestController
@RequestMapping("/api/reply")
public class ReplyController {
    @Autowired private ReplyService replyService;
    @Autowired private MailService mailService;
    @Autowired private UserService userService;

    @PostMapping("/generate")
    public ResponseEntity<?> generate(@RequestBody GenerateRequest req,
                                      Principal principal) {
        User u = userService.byEmail(principal.getName());
        MailDto mail = mailService.getById(u, req.getMessageId());
        Preference p = userService.preferenceOf(u, req.getTone());
        Reply r = replyService.generate(u, mail, p, req.getExtra());
        return ResponseEntity.ok(Map.of("replyId", r.getId(),
                                        "draft", r.getDraft()));
    }

    @GetMapping("/history")
    public List<Reply> history(Principal principal) {
        User u = userService.byEmail(principal.getName());
        return replyService.historyOf(u);
    }
}
```

LOGIN PAGE (login.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Smart Reply - Login</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <main class="auth-box">
    <h1>Login</h1>
    <form id="loginForm">
      <label for="email">Email</label>
      <input type="email" id="email" required>
      <label for="password">Password</label>
      <input type="password" id="password" minlength="8" required>
      <button type="submit">Login</button>
      <p id="msg"></p>
    </form>
    <p>New user? <a href="register.html">Create an account</a></p>
  </main>
  <script src="js/app.js"></script>
</body>
</html>
```

FRONT-END SCRIPT (app.js)

```
document.getElementById("loginForm")?.addEventListener("submit", async e => {
  e.preventDefault();
  const res = await fetch("/api/auth/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      email: document.getElementById("email").value,
      password: document.getElementById("password").value
    })
  });
  if (res.ok) {
    const data = await res.json();
    localStorage.setItem("token", data.token);
    window.location.href = "dashboard.html";
  } else {
    document.getElementById("msg").textContent = "Invalid email or password";
  }
});

async function generateReply(messageId, tone) {
  const res = await fetch("/api/reply/generate", {
    method: "POST",
    headers: { "Content-Type": "application/json",
               "Authorization": "Bearer " + localStorage.getItem("token") },
    body: JSON.stringify({ messageId: messageId, tone: tone })
  });
  if (!res.ok) { alert("Could not generate reply"); return; }
  const data = await res.json();
  document.getElementById("draft").value = data.draft;
}
```

i) SAMPLE REPLY 1 (ILLUSTRATIVE)

INCOMING EMAIL:

```
From: Prof. Meena
Subject: Project review date
Message: Could you please confirm whether your team will be ready for
the project review on Friday at 10 AM?
```

SETTINGS: Tone = Formal

OUTPUT (GENERATED DRAFT):

```
Dear Professor Meena,

Thank you for your message. I confirm that our team will be ready for the
project review on Friday at 10 AM. Please let me know if any additional
documents are required before the review.

Regards,
Ram Kumar
```

ii) SAMPLE REPLY 2 (ILLUSTRATIVE)

INCOMING EMAIL:

```
From: Arun (Client)
Subject: Quick call this week?
Message: Hi, can we have a quick call on Wednesday evening to discuss
the website changes?
```

SETTINGS: Tone = Friendly, Extra instruction = “decline politely and suggest Thursday”

OUTPUT (GENERATED DRAFT):

```
Hi Arun,

Thanks for reaching out! Wednesday evening is a little tight for me, but
I would be happy to talk on Thursday. Please let me know a time that
suits you and we can go through the website changes together.

Best wishes,
Ram Kumar
```

REFERENCES

1. K. Kannan, P. Kurach, S. Ravi, T. Kaufmann, A. Tomkins, B. Miklos, G. Corrado, L. Lukács, M. Ganea, P. Young and V. Ramavajjala, “Smart Reply: Automated Response Suggestion for Email,” in Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), San Francisco, USA, 2016, pp. 955–964.

2. M. Henderson, R. Al-Rfou, B. Strope, Y. Sung, L. Lukács, R. Guo, S. Kumar, B. Miklos and R. Kurzweil, “Efficient Natural Language Response Suggestion for Smart Reply,” arXiv preprint arXiv:1705.00652, 2017.

3. M. X. Chen, B. N. Lee, G. Bansal, Y. Cao, S. Zhang, J. Lu, J. Tsay, Y. Wang, A. M. Dai, Z. Chen, T. Sohn and Y. Wu, “Gmail Smart Compose: Real-Time Assisted Writing,” in Proc. 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), Anchorage, USA, 2019, pp. 2287–2295.

4. I. Sutskever, O. Vinyals and Q. V. Le, “Sequence to Sequence Learning with Neural Networks,” in Advances in Neural Information Processing Systems (NIPS), vol. 27, 2014, pp. 3104–3112.

5. S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

6. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, “Attention Is All You Need,” in Advances in Neural Information Processing Systems (NIPS), vol. 30, 2017, pp. 5998–6008.

7. J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in Proc. NAACL-HLT, Minneapolis, USA, 2019, pp. 4171–4186.

8. T. B. Brown et al., “Language Models are Few-Shot Learners,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 1877–1901.

9. L. Ouyang et al., “Training Language Models to Follow Instructions with Human Feedback,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 35, 2022, pp. 27730–27744.

10. P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459–9474.

11. T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” in Proc. 2020 Conference on Empirical Methods in Natural Language Processing: System

Demonstrations (EMNLP), 2020, pp. 38–45.

12. D. Hardt (Ed.), “The OAuth 2.0 Authorization Framework,” RFC 6749, Internet Engineering Task Force (IETF), Oct. 2012.
13. M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force (IETF), May 2015.
14. N. Provos and D. Mazières, “A Future-Adaptable Password Scheme,” in Proc. USENIX Annual Technical Conference (FREENIX Track), Monterey, USA, 1999, pp. 81–91.
15. R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, Dept. of Information and Computer Science, University of California, Irvine, 2000.